Système d'exploitation
CM7 : Entrées-sorties standard, bufferisation, tubes

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

mars 2026

Entrées-sorties standard

Tampons d'entrée et de sortie

Lire et écrire sur le terminal

Tubes

Redirections : `dup`

Entrées-sorties standard

Tampons d'entrée et de sortie

Lire et écrire sur le terminal

Tubes

Redirections : `dup`

stdio.h

- Sur un système Unix, `stdio.h` est une couche au-dessus des appels systèmes `read`, `write`, etc.
- On ne manipule pas de descripteur de fichier directement mais des pointeurs `FILE *`, où `FILE` est un type opaque, qui contient notamment
  - le descripteur de fichier
  - le type d'ouverture du fichier (lecture, écriture, ...)
  - un tampon (en anglais *buffer*), alloué sur le tas, permettant de minimiser le nombre de lectures et d'écritures.
  - (si pertinent) la position dans le fichier/dans le tampon.
- Les 3 flux `FILE *` déjà ouverts sont `stdout` (sortie standard), `stderr` (sortie standard des erreurs) et `stdin` (entrée standard).

fopen

```
FILE *fopen(const char *restrict chemin, const char *restrict mode);
```

- `fopen` ouvre un fichier (sur un système de type Unix, appellera `open`) dans un mode précisé dans une chaîne de caractère (pourquoi pas...)
- Le mode vaut, en résumé :
  - `"r"` qui correspond à `O_RDONLY`
  - `"w"` qui correspond à `O_WRONLY | O_CREAT | O_TRUNK`
  - `"a"` qui correspond à `O_WRONLY | O_CREAT | O_APPEND`
- retourne le flux `FILE *` créé, ou bien `NULL` en cas d'échec et `errno` est positionné (même remarque que pour `open`, toujours tester).

fclose

```
int fclose(FILE *flux);
```

Lorsqu'on a fini d'utiliser un flux, il faut le `fclose` pour :

- `close` (sur Unix) le descripteur de fichier associé
- si besoin écrire les données de sorties restantes dans le *buffer* associé
- libérer le *buffer* associé (et toutes les ressources associées au flux).
- Noter que `fclose` peut provoquer un appel système `write`.

Lire/écrire un octet à la fois

```
int fgetc(FILE *stream);
int getchar(void); // stdin
int ungetc(int c, FILE *stream);
int fputc(int c, FILE *stream);
int putchar(int c); // stdout
```

- Remarquer `int`, au lieu de `char`.
- L'`int` retourné (ou passé en argument) contient
  - dans son octet de poids faible l'octet lui-même (représenté donc ici par un `int` positif ou nul)
  - ou bien la constante `EOF` qui vaut -1.
- Remarquer la puissante magie (voir le TD) `ungetc` qui permet de faire comme si on « remet » un caractère lu dans le flux, très utile pour l'analyse lexicale.
- Terminologie : on *get* un caractère depuis un flux on *put* un caractère vers un flux.

Lire/écrire une ligne à la fois

```
char *fgets(char *s, int size, FILE *restrict stream);
int fputs(const char *restrict s, FILE *restrict stream);
int puts(const char *s); // stdout
```

- Avec `fgets`, lire au plus `size - 1` caractères ou jusqu'à la fin de la ligne (ou du fichier). Si un caractère de fin de ligne est lu, il est stocké dans la chaîne. Un caractère nul est écrit après le dernier caractère écrit dans `s`.

Lire/écrire des octets

```
size_t fread(void *ptr, size_t size, size_t nmemb,
             FILE *restrict stream);
size_t fwrite(const void *ptr, size_t size, size_t nmemb,
             FILE *restrict stream);
```

- `fread` lit au maximum `size` octets et les copie à l'adresse `ptr` ;
- `fwrite` écrit `size` octets depuis l'adresse `ptr`.

Entrées-sorties formatées

```
int printf(const char *restrict format, ...);
int fprintf(FILE *restrict stream,
            const char *restrict format, ...);
int scanf(const char *restrict format, ...);
int fscanf(FILE *restrict flux,
           const char *restrict format, ...);
```

Entrées-sorties standard

Tampons d'entrée et de sortie

Lire et écrire sur le terminal

Tubes

Redirections : `dup`

Asynchronisme

- Les lectures et écritures (appels système `read` et `write`) sous-jacents n'ont en général pas lieu lors de l'appel des fonctions précédentes.
- Il y a 3 types de `FILE *` :
  - *fully buffered* : un tampon (*buffer*) de taille fixe est utilisé pour la lecture ou l'écriture. Par exemple pour la lecture, ce tampon pourrait n'être rempli que lorsque tous ses caractères ont été consommés par des `get`. C'est le cas par défaut pour les fichiers normaux.
  - *line buffered* : le tampon n'est écrit que lorsqu'il contient une fin de ligne. C'est le cas pour l'écriture sur le terminal (`stdout`).
  - *unbuffered* : les `put` et `get` sont écrits/lus directement. C'est le cas pour l'écriture sur la sortie standard des erreurs (`stderr`).

Asynchronisme, cas *fully buffered*

Dans le cas *fully buffered*,

- les `put` dans le flux ont lieu dans un *buffer* (par exemple, de 4096 octets)
- et les écritures côté noyau (appel système `write`) ont lieu *par bloc* (par exemple, quand le tampon est plein)
- ou à la fermeture du flux (ou la fin du processus).
- Pour de la lecture, le *buffer* est (alloué et) rempli avec un `read` au premier `get` sur le flux,
- puis de nouveau, si l'on a atteint la fin du *buffer* et qu'on continue à `get`.

Asynchronisme, cas *line buffered*

Dans le cas *line buffered*,

- les `put` dans le flux ont lieu dans un *buffer* (par exemple, de 1024 octets)
- et les écritures physiques (appel système `write`) ont lieu quand le tampon est plein ou qu'une fin de ligne (`'\n'`) est `put`
- ou à la fermeture du flux (ou la fin du processus).
- Il n'y a pas de *line buffering* pour de la lecture. Dans le cas d'un `get` ou `read` sur le terminal, c'est le *terminal driver* qui s'assure que les données sont transmises pour lecture seulement lorsqu'une fin de ligne est entrée.

Quelques façons de contrôler la *bufferisation*

- `setvbuf` permet de changer la *bufferisation* associée au flux `f`, par exemple avec

  ```
  setvbuf(f1, NULL, _IONBF, 0); // non bufferisé
  setvbuf(f2, NULL, _IOFBF, 0); // fully buffered
  setvbuf(f3, NULL, _IOLBF, 0); // line buffered
  ```

  (les arguments mis à `NULL` ou `0` permettent de choisir manuellement un `buffer`, mais c'est déconseillé).
- `int fflush(FILE *flux);` force l'écriture d'un tampon
- Pour la lecture dans le terminal, il est parfois utile de vider le tampon en lisant tous les caractères restants jusqu'à la fin de ligne (ou du fichier).

  ```
  int c;
  scanf("%c", &c); // lecture d'un caractère sans sauter les blancs
  // il reste des choses dans la ligne (au moins '\n')
  while ((c = getchar()) != '\n' && c != EOF)
      ;
  ```

Entrées-sorties standard

Tampons d'entrée et de sortie

Lire et écrire sur le terminal

Tubes

Redirections : `dup`

Lignes d'entrée du terminal

Sur Linux, `man termios` nous dit :

> *La longueur maximale de ligne est 4096 caractères (y compris le caractère saut de ligne terminal) ; les lignes de plus de 4096 caractères sont tronquées. Après 4095 caractères, le traitement de l'entrée (...) se poursuit, mais les données entrées après 4095 caractères jusqu'à (mais sans l'inclure) un saut de ligne terminal sont abandonnées. Cela assure que le terminal peut toujours recevoir plus d'entrées jusqu'à ce que au moins une ligne puisse être lue.*

Mais cette limite de 4096 caractères dépend du système, donc ne pas se reposer dessus... Par exemple

```
char buffer[4096];
scanf("%s", buffer);
```

est sûr, sur Linux, mais pas forcément sur d'autres Unix. De plus, si l'entrée standard est redirigée, on ne lit plus sur le terminal...

Sorties sur le terminal

- Plus simple, c'est simplement *line-buffered*
- à ceci prêt qu'une lecture sur le terminal force l'écriture du tampon de sortie avant la lecture.
- Cela permet par exemple d'écrire :

```
int n;
// pas une ligne, donc devrait rester dans le tampon
printf("Entrer le nombre d'étoiles : ");
scanf("%d", &n); // lecture, on écrit d'abord le tampon
```

Entrées-sorties standard

Tampons d'entrée et de sortie

Lire et écrire sur le terminal

Tubes

Redirections : `dup`

Exemples dans le terminal (1)

```
$ printf '%s\n' /usr/include/*.h | grep '/std' | wc -l
```

- `printf '%s\n' /usr/include/*.h` afficher tous les fichiers du répertoire `/usr/include/` dont le nom se termine par `.h`, à raison d'un par ligne
- `grep '/std'` ne garder que les lignes qui contiennent `/std`
- `wc -l` afficher le nombre de lignes

Ce sont les deux tubes `|` qui font que

- `grep` lit la sortie de `printf` et
- `wc` lit la sortie de `grep`.

Exemples dans le terminal (2)

```
$ find . -name '*.tex' | sed 's!.*/\(.*\)!\1!g' | sort
```

- `find . -name '*.tex'` afficher les noms de tous les fichiers dans l'arborescence issue du répertoire courant dont le nom de base se termine par `.tex`
- `sed 's!.*/\(.*\)!\1!g'` pour chaque ligne d'entrée, ne conserver que ce qui suit un `/` à la fin
- `sort` trier (ici alphabétiquement) l'entrée.

Parenthèse xargs

- `basename` fait déjà ce qu'on a accompli avec notre commande `sed`

  ```
  $ basename foo/bar/baz.txt
  baz.txt
  ```

- Le problème est qu'elle ne lit pas son entrée, mais utilise ses arguments. Mais il y a `xargs` qui, d'après son `man` :

  ```
  xargs - Construire et exécuter des lignes de commandes
          à partir de l'entrée standard
  ```

- `$ printf 'foo/bar.txt\nfoo/baz.txt' | xargs basename -a`

  ```
  bar.txt
  baz.txt
  ```

  J'ai utilisé l'option GNU non standard `-a` pour que `basename` accepte plusieurs arguments... Un jour je paierai pour mes crimes.

- `$ find . -name '*.tex' | xargs basename -a | sort`

  Trouver tous les fichiers `.tex`, les donner comme arguments à `basename` et trier le résultat.

- Dernière parenthèse, sans `xargs` et avec `basename` portable, utiliser l'action `exec` de `find` :

  ```
  $ find . -name '*.tex' -exec basename {} \; | sort
  ```

Philosophie Unix

Idée principale de la philosophie Unix :

- Chaque commande fait une seule chose simple et la fait bien ;
- et le système permet l'interaction entre ces commandes ;
- ce sont ces interactions qui rendent le système puissant.

En général, ceci est renforcé par :

- l'utilisation de flux d'octets comme interface universelle (une commande lit un flux d'octets et écrit un flux d'octets) ;
- la conception de commandes qui ne font pas d'hypothèse sur l'origine de leur entrée (qui peut être le terminal, un fichier normal, ou bien la sortie d'une autre commande).

Tube Unix

- Un tube (*pipe*) est un fichier pour la communication inter-processus sur une même machine.
- Un tube est unidirectionnel, il y a une extrémité pour y écrire des données et une autre pour y lire des données.
- En général, un ou plusieurs processus (ou *threads*) écrivent dans le tube et un seul autre lit dans le tube.
- Il y a deux types de tubes : les tubes anonymes, créés avec l'appel système `pipe` et les tubes nommés (qui apparaissent dans l'arborescence des fichiers), créés par l'appel système `mkfifo`.
- `FIFO` (*first in, first out*) est un autre nom courant pour les tubes, car en effet, les octets écrits dans le tube sont dans une file d'attente : les premiers octets écrits sont les premiers qui sont lus.

Exemple minimal en C

```
void crier(int fdin, int fdout);
void repeter(int fdin, int fdout);
int main(void)
{
    int tube[2];
    if (pipe(tube) < 0) {
        perror("pipe"); exit(1);
    }
    switch (fork()) {
        case -1: perror("fork"); exit(1);
        case 0:
            close(tube[1]);
            crier(tube[0], 1);
            close(tube[0]); break;
        default:
            close(tube[0]);
            repeter(0, tube[1]);
            close(tube[1]);
            wait(NULL);
    }
    return 0;
}
```

Crier et répéter

```
void crier(int fdin, int fdout)
{
    ssize_t n;
    char buf[1024];
    while ((n = read(fdin, buf, sizeof(buf))) > 0) {
        for (size_t i = 0; i < n; i++)
            buf[i] = toupper(buf[i]);
        write(fdout, buf, n);
    }
}
void repeter(int fdin, int fdout)
{
    ssize_t n;
    char buf[1024];
    while ((n = read(fdin, buf, sizeof(buf))) > 0) {
        write(fdout, buf, n);
        write(fdout, buf, n);
    }
}
```

Tubes anonymes

- Pour créer un tube :

```
int tube[2];
if (pipe(tube) < 0) { /* gérer l'erreur */ }
```

- Extrémité `0` du tube pour la lecture : ici lire dans `tube[0]` (moyen mnémotechnique, c'est comme l'entrée standard, où on lit aussi)
- Extrémité `1` du tube pour l'écriture : ici écrire dans `tube[1]` (moyen mnémotechnique, c'est comme la sortie standard, où on écrit aussi)
- Chaque processus doit fermer (`close`) l'extrémité non utilisée du tube.
- Une fois l'écriture terminée pour toujours, il faut fermer cette extrémité, c'est souvent la seule façon pour le processus qui lit de savoir qu'il n'y a plus rien à lire dans le tube.

Lectures dans le tube et lignes

- Lire dans un tube est particulier :
  - La lecture est bloquante, tant qu'il n'y a pas d'octets à lire.
  - La lecture ne respecte pas les bords des messages : pas de correspondance entre les `write` dans côté et les `read` de l'autre.
- Il faut donc respecter une certaine forme (légère) de protocole, le plus souvent sous Unix, on échange des lignes, terminées par `'\n'`, avec une longueur maximale (POSIX impose 2048) [1].
- Ceci dit, il est possible de lire des lignes de taille arbitrairement grandes (en allouant si besoin dynamiquement plus de mémoire).
- Remarque : les lignes du terminal ont une taille maximale de toute façon (sur Linux, `man termios` indique 4096, symbole de fin de ligne compris).

1. voir `limits.h` dans https ://pubs.opengroup.org/onlinepubs/9799919799/

Écritures dans le tube

- L'appel système `write` est également bloquant si le tube est plein, ce qui arrive si le processus qui doit lire (et consommer les données) met trop de te temps à le faire.
- Attention, écrire dans un tube dont les extrémités de lecture sont toutes fermées entraîne la réception du signal `SIGPIPE` qui par défaut tue le processus.

```
int main(void)
{
    int tube[2];
    if (pipe(tube) < 0) { perror("tube"); return 1; }
    close(tube[0]);
    printf("Je vais mourir.\n");
    write(tube[1], "a", 1);
    sleep(1000); /* interrompu par SIGPIPE */
    return 0;
}
```

```
$ ./a.out
Je vais mourir
$ kill -l $?
PIPE
```

Écriture et atomicité

- Que se passe-t-il si plusieurs processus écrivent dans le tube ?
- POSIX garantit qu'une écriture est atomique (chouette !) si sa taille ne dépasse pas `PIPE_BUF` octets.
- Valeur minimale de `PIPE_BUF` (POSIX) : 512
- Sur Linux : 4096 (`man 7 pipe`)
- Donc si les écritures ne font pas plus de 512 octets, les messages écrits par divers processus ou *threads* dans le tube ne sont pas mélangés (même si l'ordre de ces messages peut être arbitraire).

Tubes nommés

- Commande `mkfifo` pour créer un tube nommé, c'est-à-dire un fichier de type `p` dans l'arborescence.

```
$ mkfifo tube
$ ls -l tube
prw-r--r--. 1 pierre pierre 0  3 mai   16:14 tube
```

- Remarque : ce fichier ne prend pas de place sur le disque, son contenu est toujours en RAM, comme un tube anonyme.
- On peut utiliser ce tube comme un tube anonyme, mais en explicitant les redirections. Ici, le caractère `&` termine la première commande en la mettant en arrière-plan.

```
$ seq 1 100 >tube& grep [48]2 <tube
[1] 38189
42
82
[1]+  Fini                    seq 1 100 > tube
```

- Mais attention, ceci est un *deadlock* :

```
$ seq 1 100 >tube; grep [48]2 <tube
```

car l'ouverture (`open`) d'un tube nommé est bloquante jusqu'à ce qu'il soit ouvert des deux côtés (`man 7 fifo`).

Appel système mkfifo

```
void crier(int fdin, int fdout);
void repeter(int fdin, int fdout);
int main(void) {
    if (mkfifo("mon_tube", 0644) < 0) {
        perror("mkfifo"); return 1;
    }
    int fd;
    switch (fork()) {
    case -1: perror("fork"); exit(1);
    case 0:
        fd = open("mon_tube", O_RDONLY);
        if (fd < 0) { perror("open"); return 1; }
        crier(fd, 1);
        close(fd); return 0;
    default:
        fd = open("mon_tube", O_WRONLY);
        repeter(0, fd);
        close(fd);
        wait(NULL);
        unlink("mon_tube"); /* supprimer le fichier */
    }
}
```

mkfifo

- `int mkfifo(const char *chemin, mode_t mode);`
  - `chemin` pour le nom du nouveau fichier (peut être absolu ou relatif)
  - `mode` pour les permissions (voir le cours sur `open`),
  - retourne 0 en cas de succès et -1 en cas d'échec (en remplissant `errno`).
- L'ouvrir
  - en écriture dans les processus qui veulent y écrire des données,
  - et en lecture dans le processus qui y lit des données.
- `open` est bloquant tant qu'il n'y a pas un processus pour lire et au moins un autre pour écrire.
- En-dehors de ça, `read` et `write` fonctionnent exactement comme avec les tubes anonymes.

Entrées-sorties standard

Tampons d'entrée et de sortie

Lire et écrire sur le terminal

Tubes

Redirections : `dup`

find | grep | awk | sed | tr | wc -l

- Il manque un dernier outil pour comprendre comment fonctionnent les *pipe* du shell :
- la redirection, aussi appelée duplication des descripteurs de fichier.
- Ceci se fait à l'aide des appels système `dup` et `dup2`.
- ```
  #include <unistd.h>
  int dup(int oldfd);
  int dup2(int oldfd, int newfd);
  ```
- `dup` cherche le plus petit descripteur de fichier disponible, lui fait décrire le même fichier que `oldfd` et le retourne.
- `dup2` prend comme deuxième argument le nouveau descripteur pour le fichier décrit par `oldfd` ; si `newfd` désignait déjà un fichier ouvert, celui-ci est fermé.
  ```

Écrire dans un fichier avec `printf`, lire dans un fichier avec `scanf`

```c
/* echange_colonnes.c */
int main(int argc, char *argv[])
{
    int fin, fout;
    if ((fin = open(argv[1], O_RDONLY)) < 0) {
        perror("open"); return 1;
    }
    if ((fout = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644)) < 0)
        perror("open"); return 1;
    }
    char col1[2048], col2[2048];
    dup2(fin, 0); /* redirection de l'entrée standard */
    dup2(fout, 1); /* redirection de la sortie standard */
    close(fin); close(fout); /* plus besoin d'eux */
    while (scanf("%2047s %2047s", col1, col2) == 2) {
        printf("%s %s\n", col2, col1);
    }
}
```

Même chose avec dup

```
int main(int argc, char *argv[])
{
    int fin, fout;
    if ((fin = open(argv[1], O_RDONLY)) < 0) {
        perror("open"); return 1;
    }
    if ((fout = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644)) < 0)
        perror("open"); return 1;
    }
    char col1[2048], col2[2048];
    close(0); /* 0 est maintenant le plus petit fd dispo */
    dup(fin); /* 0 désigne le même fichier que fin
             c'est donc une redirection de l'entrée standard */
    close(1); /* 1 est maintenant le plus petit fd dispo */
    dup(fout); /* 1 désigne le même fichier que fout
             c'est donc une redirection de la sortie standard */
    close(fin); close(fout); /* plus besoin d'eux */
    while (scanf("%2047s %2047s", col1, col2) == 2) {
        printf("%s %s\n", col2, col1);
    }
}
```

Pourquoi ça marche ?

- Beaucoup de fonctions (`printf`, `putchar`, `puts`, ...) écrivent directement dans le descripteur `1`, c'est-à-dire la sortie standard ;
- après l'appel à `dup2(fd, 1)`, le descripteur `1` désigne maintenant le fichier décrit par `fd`, et ces fonctions se retrouvent, sans le savoir (ou presque...), à écrire dans un autre fichier que le terminal.
- Idem pour l'entrée standard (`0`) dans laquelle lisent les fonctions comme `scanf`, `getchar`, ...
- après `dup2(fd, 0)`, ces fonctions lisent en fait dans le fichier décrit par `fd`.

Tuyauterie

```
int tube1[2]; pipe(tube1);
if (fork() == 0) {
  close(tube1[0]);
  dup2(tube1[1], 1); close(tube1[1]);
  execlp("find", "find", ".", "-name", "*.txt", NULL);
}
close(tube1[1]);
int tube2[2]; pipe(tube2);
if (fork() == 0) {
  close(tube2[0]);
  dup2(tube1[0], 0); close(tube1[0]);
  dup2(tube2[1], 1); close(tube2[1]);
  execlp("sed", "sed", "s!.*/\\(.*\\)!\\1!g", NULL);
}
close(tube1[0]); close(tube2[1]);
if (fork() == 0) {
  dup2(tube2[0], 0); close(tube2[0]);
  execlp("sort", "sort", NULL);
}
close(tube2[0]);
while (wait(NULL) != -1)
  ;
```